

```

{
    public delegate void EventDelegate(EventBasics e);
    public event EventDelegate OnVarChange;
    int var;
    public int Var
    {
        get { return var; }
        set { var = value; OnVarChange(this); }
    }
    public void PrintVar(EventBasics e) { Console.
WriteLine(e.Var); }
    public static void Test()
    {
        EventBasics b = new EventBasics();
        // Note PrintVar is not static so we must access
it through an object
        // of type EventBasics; implicitly, "this" will
be this object.
        b.OnVarChange += b.PrintVar;
        b.Var = 15;
        b.Var = 20;
    }
}

```

This code should look awfully familiar. If you have a good understanding of what delegates provide, it looks like that the event declaration is not adding any extra functionality. That is, we could replace the event member `OnVarChange` with a delegate field `OnVarChange` and get the same effect.

In some sense, events provide a similar abstraction between delegates and the notion of an event that properties provide between instance variables and the notion of field access. The event above automatically provides a backing field of type `EventDelegate` that powers the event. Alternatively, we can provide implementations of the essential functionality of the event abstraction: adding/registering and removing/unregistering.

In the example below, we create an event that double-adds the delegates passed to it. You can imagine writing a custom event that conditionally adds to different delegates or providing locking/synchronization support for an event.

```
public class SpecializedEvent
```